

GridLedger

A National Electricity Billing Engine

Object-Oriented Programming — Standalone Lab

Marks: 10 **Language:** Java (JDK 17) **Approach:** Repair an existing codebase, test-driven
You are given a working program and a test file. Submit your .java source files and do not modify the test file.

The job

The National Power Distribution Company bills every metered connection in the country once a month. A contractor was paid to build the billing engine; they delivered something that *produces the right numbers* and passes the company's acceptance tests — and is, by every other measure, a disaster to live with. You have been brought in to rebuild it so the company can actually maintain it. The numbers must stay exactly right. What changes is the *shape* of the code that produces them.

This is a repair job, not a green field

You are handed two things: the contractor's source, and the acceptance test file the company signs off against. The contractor's code **compiles and passes the company's business tests today** — a residential bill comes out correct to the paisa, an industrial bill likewise. If all you cared about was this month's numbers, you could ship it and go home.

But the company does not bill once. It bills every connection, every month, forever, and the tariffs *change* — a new customer category is created by the regulator, a slab boundary moves, a subsidy is announced in the budget. Every one of those ordinary events turns the contractor's code into a minefield. Here is the whole engine as the contractor wrote it:

```
class Bill {
    String customerType;      // "RESIDENTIAL", "COMMERCIAL", "INDUSTRIAL"
    int previousReading;
    int currentReading;
    boolean subsidised;       // a residential flag bolted on later

    double energyCharge() {
        int units = currentReading - previousReading;
        if (customerType == "RESIDENTIAL") {
            double c = 0;
            if (units <= 50)      c += units * 4.0;
            else { c += 50 * 4.0;
                if (units <= 200) c += (units-50) * 7.0;
                else { c += 150 * 7.0; c += (units-200) * 11.0; } }
            if (subsidised) c = c - (c * 0.30);      // lifeline rebate
            return c;
        } else if (customerType == "COMMERCIAL") {
            double c = 0;
            if (units <= 100)     c += units * 9.0;
            else { c += 100 * 9.0; c += (units-100) * 13.0; }
            return c;
        } else if (customerType == "INDUSTRIAL") {
            // ... yet another slab ladder ...
        }
        return 0;
    }
}
```

```

double fixedCharge() { if (customerType == "RESIDENTIAL") ... else if
    ... }
double fuelSurcharge(){ if (customerType == "RESIDENTIAL") ... else if
    ... }
double tax()          { if (customerType == "RESIDENTIAL") ... else if
    ... }
double total()        { /* energy + fixed + fuel + tax, each re-branching
    */ }
}

```

It worked on delivery day. Then the company tried to *use* it, and every method above became a place where things go wrong:

The contractor's version, and why it hurts

- **Every calculation asks the same question first.** Before `energyCharge` can do anything, it must ask “*what kind of customer is this?*” So must `fixedCharge`, `fuelSurcharge`, `tax`, and `total`. The very same `if/else` ladder on `customerType` is copied into five methods, and they drift: someone fixes a rate in `energyCharge` and forgets the twin branch in `fuelSurcharge`.
- **Adding a category means editing everything.** The regulator introduces an “Agricultural” tariff. There is no one place to add it — you must open every method and thread one more `else if` through each, in the right order, five times, without a mistake.
- **The slab arithmetic is nested three deep.** Look at the residential branch: a customer-type check, wrapped around a slab check, wrapped around another slab check. Reading it, nobody can say out loud what the second slab costs without tracing every brace.
- **The subsidy was bolted on with a flag.** “Lifeline” customers are ordinary residential customers who receive a rebate. The contractor added a boolean `subsidised` and an `if (subsidised)` *inside* the residential branch. Now a residential bill has two shapes depending on a flag, the rebate logic lives tangled in the middle of the slab maths, and a subsidised commercial connection — a thing that cannot exist — is still a perfectly legal object.
- **A half-built bill is a legal object.** `new Bill()` with no `customerType` set compiles, and then every method silently returns `0`. A connection with a `currentReading` *below* its `previousReading` — a meter that ran backwards — sails through and bills a negative number of units.

Your task is to rebuild `GridLedger` so that none of the five pains above can happen again, while the bills come out to the same numbers. **Nothing in this document tells you which Java features to use.** It tells you how the billing world behaves; the shape of your classes is the thing you are being marked on. But read the five pains as a promise: if your rebuild keeps the same shape as the contractor's, it will hit the same five walls — and the test file will catch it at the first one.

How the billing world behaves

Read these as facts about the business. Each one quietly decides something about how your classes must relate. The *italic* phrases and the **bold** phrases are load-bearing — they are the hints about which things are the same underneath and which things merely differ on the surface.

The connection and its meter

- Every billed thing is a **connection**: a customer, and the electricity meter serving them. A connection *has* a meter — it plainly is not itself a meter, and a meter is not a customer; one simply *holds* the other and asks it for its readings when it is time to bill.

- A meter is read once at the start of the period and once at the end. The **units consumed** are the difference between the two readings; this number is *worked out from the readings on demand*, never typed in and never stored as a separate figure that someone has to keep in step.
- A meter never runs backwards. *A closing reading below the opening reading is not a cheap month; it is a broken record*, and the system must refuse it rather than bill a negative quantity. A reading itself can never be negative.

What every bill is made of

Whatever the customer's category, a month's bill is always the sum of the same four parts, *added up the same way for everyone*:

1. an **energy charge** for the units consumed, worked out against that category's slab rates;
2. a flat **fixed charge** for having the connection at all;
3. a **fuel-price surcharge**, a percentage of the energy charge that the regulator sets each month for the whole country;
4. **tax**, a percentage added on the sum of the three parts above.

Notice the shape of that list. *How* the energy charge is worked out is the one thing that genuinely differs from category to category — everything else (adding the four parts, applying the country-wide fuel percentage, applying tax) is done **identically for every customer in the land**. Whatever is truly common should be *written down once, in one place*, and the categories should differ *only where they actually differ*. A design that re-states the “add up the four parts and apply tax” logic separately inside every category has copied out loud something that was always the same sentence.

The customer categories — where the behaviour splits

There is no such thing as a “plain” connection. *Every connection is of a definite category*, and the category is not a label you read off a field — it **is what the connection fundamentally is**, and it is the category that decides how the energy charge is worked out:

- A **residential** connection is charged on a rising three-band slab: the first block of units at a low rate, the next block dearer, everything beyond that dearer still. Its fixed charge is small.
- A **commercial** connection is charged on a two-band slab at higher rates, with a larger fixed charge.
- An **industrial** connection is charged at a single flat rate per unit for all consumption, but carries the largest fixed charge of all.

Each category answers the question “what is the energy charge for these units?” *in its own way*, and yet the company must be able to treat any connection as simply “a connection to be billed” when it does not care which category it is (see the billing run below). *These three are not three unrelated classes and they are not one class with a label* — they are three **particular kinds of the same underlying thing**, alike in everything the previous section listed and different only in the one calculation named here.

▷ Notice

This is the heart of the lab. When the regulator adds a fourth category next year, there must be exactly **one** place to describe it — its own slab rule and its own fixed charge — and *nothing else in the system should have to be reopened*. In particular, the billing run, the total, and the fuel and tax logic must not contain any list of categories that would need a new entry. If adding a category makes you edit a method that computes a total, your rebuild has the contractor's

shape.

The lifeline (subsidised) connection — built on, not bolted on

- A **lifeline** connection is the government’s subsidy for low-income households. It is *not a fourth category*, and it is *not a flag*. **A lifeline connection is a residential connection** — it consumes on exactly the residential slab, it carries the residential fixed charge, it is taxed like any residential connection — with one thing added: a fixed-percentage **rebate taken off whatever the residential energy charge already came to**.
- This is the rule the contractor could not express with a boolean. If the residential slab rates are ever changed, the lifeline bill must change *with them, for free*, without anyone re-typing a single rate. *The lifeline connection must build on the residential one, taking its energy charge and reducing it* — never restating the residential slab for itself.
- Because a lifeline connection **is a residential connection**, the impossible cases the contractor allowed — a “subsidised industrial” connection, a “subsidised” anything-but-residential — simply *cannot be written down* in your design. The subsidy exists only where it is a residential connection to begin with.

The billing run (the “cycle”)

- The monthly billing run holds a batch of connections — residential, commercial, industrial, lifeline, all together in *one* place. From the run’s point of view they are simply “connections to be billed,” and it must be able to **hold any of them side by side** without caring which category each one is.
- **Bill the whole batch in one sweep.** Running the cycle produces an invoice for *every* connection it holds in a single pass — “for each connection waiting to be billed, work out its bill” — and hands back the set of invoices. That loop *must not contain a ladder that asks each connection which category it is*; each connection must already **know how to bill itself**.
- **Total the run in one sweep.** The run can report the grand total it will collect this month, again by walking the batch once and asking each connection for its bill total — with *no per-category branching in the walk*.
- **One name, two ways to register.** Registering a connection into the run should be possible either on its own, or together with the month’s fuel-surcharge percentage that applies to it. Same idea, same name, different inputs — the caller should not have to remember two different method names for “add this connection to the run.”
- The run can also say how many connections are currently in the batch.

Decide the relationships before you type. Every bold and italic phrase above — “has a meter,” “worked out on demand,” “written down once,” “in its own way,” “particular kinds of the same underlying thing,” “a lifeline connection is a residential connection . . . built on,” “hold any of them side by side,” “know how to bill itself” — is describing a relationship between two of your classes. Work out, on paper, which classes those are and how they connect *before* you open the editor. Students who start by typing classes almost always discover halfway through that two of them should have been one, or that one of them should have been built on another.

The slab arithmetic, worked once so you are sure of it

The one piece of real arithmetic in this lab is the rising slab. It trips people up, so here it is spelled out. A slab charges *the first block of units at one rate, the next block at another*, and so on — **not** every

unit at the rate of the band it lands in.

Residential example (bands at 50 and 200 units). First 50 units at 4.0, the next 150 (from 51 to 200) at 7.0, everything above 200 at 11.0. So for **320 units**:

$$\underbrace{50 \times 4.0}_{200.0} + \underbrace{150 \times 7.0}_{1050.0} + \underbrace{(320 - 200) \times 11.0}_{120 \times 11.0 = 1320.0} = 2570.0$$

For **40 units** (inside the first band): $40 \times 4.0 = 160.0$, nothing more. For **120 units**: $50 \times 4.0 + (120 - 50) \times 7.0 = 200.0 + 490.0 = 690.0$. The exact band boundaries and rates for each category are fixed in the test file — read them there; the shape of the sum is what matters here.

▷ Notice

The contractor’s nested ifs exist *only* because the slab and the category were tangled together. Once each category owns its own energy-charge rule, a single category’s slab is a short, flat calculation — “fill the first band, then the next, then the rest” — with no customer-type question wrapped around it. If your rebuilt residential calculation is still three braces deep, ask what the outer brace is testing, and whether that test still needs to exist at all.

Step 0 — Sketch the interactions (do this first, and submit it)

Before any code, draw — by hand is fine — how these objects hand work to one another when the monthly run is executed. Show, for example: who the billing run talks to when you execute it; what a residential connection has to ask before it can produce its energy charge; what the lifeline connection leans on to do its job; where the meter comes in and who owns it. Use boxes and arrows and your own labels; **this is not UML and needs no formal notation** — a clear informal picture is exactly what we want. Photograph or scan it and submit it with your code. You will be asked to talk through it.

Question Your Viva Will Ask (the sketch)

Point at the single arrow in your drawing that does the real work when the run is executed — the one call that, repeated over the batch, somehow produces a three-band slab charge for one connection and a flat industrial charge for the next. Who decides which of those two calculations actually happens, and at what instant: when you wrote the loop, or while the loop is running? If your drawing has a separate arrow per category, you have drawn the contractor’s version again.

What each class must expose

Here is the catalogue of behaviours your classes must offer, by *name and effect only*. Notice what is deliberately missing: we are **not** telling you any method’s parameters or its return type, and we are not telling you which class should sit above which. You have done test-driven development before; the provided test file is your specification for the exact signatures, and the relationships are yours to design from the rules above.

- **A meter** can be asked for its opening and closing readings and for the units consumed. It refuses a reading that is negative or a closing value below the opening value.
- **Every connection**, whatever its category, can:
 - report its *energy charge* — worked out on its own category’s slab;
 - report its *fixed charge*;

- report its *fuel surcharge* and its *tax* (these follow the same country-wide rules for all categories);
- report its *bill total* — the four parts added up.

A connection with no category cannot answer any of these, and must not be constructible.

- **A residential connection, a commercial connection, and an industrial connection** each provide the energy charge *in their own way*, per the category rules; everything common they share *without restating it*.
- **A lifeline connection** provides them too — but its energy charge is built *from* the residential one (the residential charge, less the rebate), not written afresh.
- **The billing run** can: take a connection to be billed (and, by the same name, take a connection together with the fuel-surcharge percentage that applies); *execute* the whole batch and return the set of invoices; report the batch's *grand total*; and report how many connections are waiting.

▷ Notice

The test file constructs each kind of connection *by its own name* — a residential one, a commercial one, an industrial one, a lifeline one — and then drops a mixture of them into a single billing run. A design that funnels every category through one class with a “which category am I” field will not even compile against the test, let alone pass it — and that, precisely, is the lesson. Read the test file the way you have been taught: let it tell you each method's return type and parameters; let *this* document tell you how the classes relate.

What the test file checks (in words — read the file for the types)

You will be given `GridLedgerTest.java`. You do not modify it. The following is what it verifies, described so you know what to build; the exact signatures, rates, and band boundaries are in the file.

- **The meter guards itself.** A negative reading is refused; a closing reading below the opening reading is refused; a valid pair reports the correct units consumed, computed from the two readings and not stored separately.
- **Each category charges energy in its own voice.** Feed the *same* units to one connection of each category and check three visibly different energy charges — the residential one following its three-band slab, the commercial one its two-band slab, the industrial one its flat rate.
- **The common parts are genuinely common.** For every category, the bill total equals energy + fixed + fuel-surcharge + tax, with the fuel percentage and tax percentage applied by the *same* rule regardless of category. (These are not numbers you stored; they follow from the parts.)
- **The lifeline connection is built on the residential one.** For the same units, the lifeline energy charge equals the plain residential energy charge reduced by exactly the rebate percentage; and everything else about its bill matches a residential bill. A later check changes the residential slab and confirms the lifeline charge moves with it, with no edit of its own.
- **One run, many categories, side by side.** Drop a residential, a commercial, an industrial, and a lifeline connection into one run; it reports four waiting and a grand total equal to the four individual bill totals added up by hand.
- **Executing bills everything in one sweep.** The set of invoices returned by a run contains every connection's own bill total, for every category, in the order they were registered.
- **One slot, many forms.** A single variable is pointed at a residential connection, then at a commercial one, then at an industrial one; each time it is registered and later billed correctly — the same variable standing in for any category.

- **Register plainly, or register with the fuel percentage.** Registering a connection with a fuel-surcharge percentage applies that percentage to its bill; registering it the plain way uses the default — same name, two input shapes.

Question Your Viva Will Ask (the total)

Your “grand total” walks the batch and asks each connection the *same* question under the *same* name, yet a residential, a commercial, an industrial, and a lifeline connection all return different numbers. The walk was written before any particular batch existed. *Who* picks the right energy-charge rule for each connection, and *when*? Contrast that with the two ways of registering a connection (plain, or with a fuel percentage): which of those two is chosen by the same mechanism, and which is chosen earlier, by the compiler, from the way the call was written?

Marking Rubric

#	Criterion	Marks
1	The three categories each work out their energy charge in their own way, and can be held together in one billing run through a single common type	3
2	Execute and grand-total each work as a single sweep over the mixed batch, with no per-category branch ladder inside the sweep	2
3	The lifeline connection is built <i>on</i> the residential one (its energy charge reuses the residential charge rather than copying the slab) and sits correctly in the chain above it	2
4	The four common bill parts (adding up, fuel surcharge, tax) are written once and shared, not restated per category; the meter and readings are sealed against outside tampering; a category-less connection cannot be constructed	1.5
5	Registering offers one name with two input shapes (plain, and with a fuel-surcharge percentage)	1
6	Meter modelled as something a connection <i>holds</i> , not a kind of connection; clear informal interaction sketch submitted	0.5
Total		10

▷ Notice

Passing the tests is necessary but not sufficient. Some properties cannot be written as an assertion — that a category-less or “subsidised-industrial” connection is impossible to construct, and that the common bill parts are written in exactly one place — and the is-a / has-a choices are a matter of design, not output. Criteria 3, 4, and 6 are judged by reading your code and by the viva. A solution that reaches the right numbers through one class and a forest of if/else on a category field has rebuilt the contractor’s version and will score accordingly, however green the test bar looks.

Ground rules

- Java 17. Use double for money and int for readings, units, and counts.

- Store the batch however you like — a `java.util.ArrayList` is perfectly fine, and a plain array is fine too. The container is not the point of this lab; *what* you can put in it is. Whatever you choose, the run must be able to hold every category of connection side by side under a single common type, and walk them without ever asking which category each one is.
- The numbers must not change. Every rate, band boundary, and percentage the contractor used is fixed in the test file; your rebuild must reproduce them exactly. You are changing the *shape* of the code, not the bill.
- Say every class relationship aloud before you commit to it: “*is a*” versus “*has a*.” A lifeline connection *is a* residential connection; a connection *has a* meter. Getting that split right is most of the design.
- Partial credit is awarded; submit whatever compiles. Submit your source files and your Step-0 sketch.